

RecoverBench: Supplementary Material

A Implementation Details

Validation criteria. All validators return a unified `ValidationResult_v5` comprising a boolean ok flag, the classified degree, a metrics dictionary, and a human-readable reason string. Table 1 summarizes the validation condition and degree threshold for each Error Skill.

Table 1: Validation condition and degree threshold for each Error Skill. Scenes that fail the occurrence condition are rejected; physically implausible states are implicitly filtered because they cannot satisfy these conditions.

ID	Error Skill	Error Validation Criterion	D0 / D1 Threshold
E1	grasp_misalignment	Gripper closed $\wedge$ (EEF offset $\geq 1\text{cm}$ or rotation $\geq 0.08\text{ rad}$)	D0: offset 5–40mm D1: offset + rotation 0.3–1.0 rad
E2	drop_in_transit	Separation $> 0.08\text{m} \wedge \Delta z \geq 3\text{cm}$	D0: gentle release, settle 80 steps D1: fast drop, settle 40 steps
E3	drop_at_wrong_place	Separation $> 0.08\text{m} \wedge \Delta z \geq 3\text{cm}$	D0: small offset from target D1: large offset from target
E4	collision_holding	Force $> 0.5\text{N}$ or obj. disp. $\geq 1\text{cm}$ or EEF disp. $> 5\text{cm}$	D0: downward 5–10cm D1: diagonal 8–15cm + lateral
E5	collision_empty	Non-target obj. disp. $\geq 1\text{cm}$	D0: overshoot 2–4mm D1: overshoot 4–8mm
E6	collision_eef_object	Non-target obj. disp. $\geq 0.5\text{cm}$	D0: small overshoot D1: large overshoot
E7	collision_self	Joint proximity $\geq 0.6 \wedge$ EEF disp. $> 2\text{cm}$	D0: moderate retract D1: aggressive retract
E8	wrong_object	Gripper closed $\wedge$ closer to non-target	D0: grasp only D1: grasp + lift attempt
E9	trajectory_regression	Phase index decreased or z -regression $\geq 3\text{cm}$	D0: partial regression D1: full phase regression
E10	stuck_no_progress	EEF disp. $< 0.002\text{ m/step} \times$ freeze steps	D0: 20-step freeze D1: 40-step freeze
E11	grasp_wrong_pose	Gripper closed $\wedge$ rotation $\geq 0.08\text{ rad}$	D0: rotation $\geq 0.08\text{ rad}$ D1: rotation $\geq 0.15\text{ rad}$
E12	position_error	EEF disp. $\geq 1\text{cm}$ from intended trajectory	D0: offset 3–10cm D1: offset 10–20cm + rot. 0.15–0.8 rad

Physical plausibility filters. Scenes are rejected if they violate workspace or stability constraints. Workspace filters require end-effector positions within $x, y \in [-1, 1]\text{ m}$ and $z \in [0, 2]\text{ m}$, and object positions within $x, y \in [-1, 1]\text{ m}$ and $z \in [-0.5, 2]\text{ m}$, eliminating cases where objects leave the table or penetrate geometry. Stability filters reject scenes with maximum joint velocity $> 15.0\text{ rad/s}$, maximum object linear velocity $> 2.5\text{ m/s}$, or any NaN values in the simulator state. A 5-step warm-start period after injection allows the MuJoCo integrator to stabilize before these checks are applied.

Error verification details. Drop errors (E2, E3) are confirmed by object-gripper separation greater than 0.08 m together with an object height decrease of at least 3 cm. Collision errors (E4–E7) are verified through either contact force above 0.5 N or displacement beyond the corresponding skill threshold, using 1 cm for object displacement and 5 cm for end-effector displacement. Grasp errors (E1, E11) require the gripper to be closed while the end-effector deviates from the intended grasp by at least 1 cm in position or 0.08 rad in orientation. Wrong-object errors (E8) are identified when a closed gripper is closer to a non-target object than to the intended target. For temporal and trajectory errors, we verify trajectory regression (E9) by checking whether task progress falls below $0.002\text{ m/step} \times$ freeze_steps, and position errors (E12) by requiring at least 1 cm deviation from the intended end-effector trajectory.

Environment fingerprint and reproducibility. Each error scene stores the complete post-injection simulator state (MuJoCo `qpos/qvel` vector for all joints and objects). During evaluation, this state is loaded directly via `set_sim_state_flat()`, restoring the exact post-error configuration as the episode starting point. Environment consistency is guaranteed by a state-dimension check (mismatched dimensions raise an error at load time) and an environment fingerprint (SHA-256 hash of the MuJoCo XML model) stored alongside each scene. Each error scene file (NPZ format) contains: simulation state (`qpos`, `qvel` for all joints, object positions and orientations), RNG state (NumPy random state snapshot at the moment of injection), and environment fingerprint (SHA-256 hash of the MuJoCo XML model file and robosuite configuration). Deterministic replay is verified by loading `states[0]` and replaying stored actions, confirming identical resulting trajectories.

Training hyperparameters. Table 2 summarizes the training configuration for the four evaluated policy variants. The BC-RNN policies use the same robomimic architecture and optimization setup: a two-layer LSTM policy with hidden size 1,000, sequence horizon 10, batch size 16, and Adam optimization with learning rate 1×10^{-4} and L2 regularization 1×10^{-4} . Both BC-RNN variants are trained for 50 epochs. BC-RNN (Nominal FT) is trained on nominal MimicGen demonstrations only, whereas BC-RNN (Recovery) appends successful recovery trajectories to the nominal dataset once per task.

The Pi 0.5 policies are initialized from the Pi 0.5 base checkpoint and fine-tuned using LoRA adapters applied to both the Gemma-2B vision-language backbone and the 300M action expert. The corresponding LoRA ranks are 16 and 32, respectively. Both Pi 0.5 variants use an action horizon of 10, batch size 64, and AdamW optimization with a cosine learning-rate schedule: 1,000 warmup steps, peak learning rate 5×10^{-5} , and decay to 5×10^{-6} over 20,000 steps. Pi 0.5 (Nominal FT) is trained on nominal merged demonstrations for 20,000 optimization steps. Pi 0.5 (Recovery) follows the same LoRA fine-tuning protocol but is trained for 20,000 steps on the concatenation of one copy of the nominal dataset and one copy of the recovery dataset.

Table 2: Training hyperparameters for the evaluated policy variants. For Pi 0.5, the reported LoRA ranks correspond to the Gemma-2B backbone and the 300M action expert, respectively.

Policy	Optimizer	Learning Rate	Batch Size	Training Length	Architecture / Adaptation
BC-RNN (Nominal FT)	Adam	1×10^{-4}	16	50 epochs	2-layer LSTM, hidden size 1,000, horizon 10
BC-RNN (Recovery)	Adam	1×10^{-4}	16	50 epochs	2-layer LSTM, hidden size 1,000, horizon 10
Pi 0.5 (Nominal FT)	AdamW	$5 \times 10^{-5} \rightarrow 5 \times 10^{-6}$	64	20,000 steps	Pi 0.5 base; LoRA ranks 16 / 32; horizon 10
Pi 0.5 (Recovery)	AdamW	$5 \times 10^{-5} \rightarrow 5 \times 10^{-6}$	64	20,000 steps	Pi 0.5 base; LoRA ranks 16 / 32; horizon 10

Hardware and runtime. All training and evaluation use NVIDIA A800 80GB GPUs. End-to-end training and evaluation for all policy variants takes approximately 30 hours of total GPU time.

Evaluation protocol. Rollout horizon: 500 environment steps per episode. Task success is determined by the environment’s native `_check_success()` predicate (robosuite task completion, e.g., target object at goal position), evaluated at every step; an episode terminates early upon success or after exhausting the 500-step budget. Policy inference is deterministic: the VLA server produces a 5-step action chunk per query with no stochastic sampling. Each scene is evaluated once, so the scene itself is the unit of analysis.

Statistical significance. Each error scene is evaluated once with deterministic policy inference (no stochastic sampling), so the scene itself is the unit of analysis. We report Recovery Success Rate as the fraction of successful recoveries out of total scenes per subtype, task, or aggregate. The Kruskal-Wallis H-test reported in the main paper ($H = 19.83$, $p = 0.048$) tests whether per-subtype RSR distributions differ across the 12 Error Skills. Spearman rank correlations between state displacement and RSR are reported with two-tailed p -values.

B Dataset Details

Dataset composition. Table 3 lists all 6 tasks organized into 3 difficulty tiers together with validated scene counts, source trajectory counts, and target objects per task.

Table 3: Dataset statistics. Tasks are organized into 3 difficulty tiers. The validated scenes constitute the benchmark scene set and can also seed recovery-demo collection.

Tier	Task	Error Subtypes	Validated Scenes	Clean Trajectories	Objects
1	pick_place	24	240	50	Milk, Bread, Cereal, Can
1	stack	24	240	50	Block A, Block B
2	coffee	20	200	50	Mug, Coffee pod
2	threading	20	200	50	Needle, Thread
3	stack_three	24	240	50	Block A, B, C
3	three_piece_assembly	24	240	50	3 assembly parts
Total		136	1,360	300	

Tier 1 tasks (pick_place, stack) involve single-stage pick-and-place and stacking, and expose all 12 Error Skills (24 subtypes). Tier 2 tasks (coffee, threading) introduce geometric constraints (insertion, alignment) that reduce feasibility for some Error Skills, yielding 20 subtypes. Tier 3 tasks (stack_three, three_piece_assembly) combine multi-object coordination with the full Error Skill set, re-exposing 24 subtypes. We target 10 validated scenes per feasible (task, subtype) pair, with quota reallocation when some combinations are undersubscribed after feasibility and validation filtering.

Evaluated policies. Table 4 summarizes the four policies evaluated on RECOVERBENCH.

Table 4: Evaluated policies.

Policy	Type	Training / Source	Notes
BC-RNN (Nominal FT)	Task-specific IL	MimicGen demos	robomimic LSTM trained on clean demos only
BC-RNN (Recovery)	Task-specific IL	MimicGen + recovery demos	robomimic LSTM with recovery supervision
Pi 0.5 (Nominal FT)	Task-specific VLA	MimicGen + LoRA	Fine-tuned without recovery data
Pi 0.5 (Recovery)	Task-specific VLA	MimicGen + recovery demos	Additional recovery supervision

Recovery demonstration statistics. Across all 6 tasks, 1,641 human recovery demonstrations yield 8,957 augmented trajectories via MimicGen-style subtask warping, for 10,598 total recovery trajectories. Table 5 reports the per-task breakdown.

Table 5: Recovery demonstration collection statistics per task. MimicGen refers to demonstrations generated via subtask warping from clean trajectories; Human refers to teleoperated recovery demonstrations from post-error states; Augmented refers to MimicGen-style retargeting of human recovery demos to new scenes.

Task	MimicGen	Human	Augmented	Total
stack	248	24	1,154	1,426
pick_place	223	88	14	325
coffee	47	190	1,802	2,039
threading	7	212	1,746	1,965
stack_three	55	229	2,139	2,423
three_piece_assembly	88	230	2,102	2,420
Total	668	973	8,957	10,598

MimicGen augmentation details. Recovery demonstrations are augmented via three complementary strategies, all built on MimicGen-style object-centric subtask warping:

(3A) Scene augmentation. For each source demo, ErrorSceneVariantGenerator re-samples object positions using the task’s placement_initializer with Gaussian noise

(position_noise_std=0.005 m). Each recovery trajectory is RBG-segmented into canonical subtasks (Table 6), and per-subtask object-centric warping transforms the source EEF poses to the new scene geometry via $T_{\text{relative}} = T_{\text{target_obj}} \cdot T_{\text{src_obj}}^{-1}$. Warped waypoints are executed through MimicGen’s WaypointTrajectory interface with success validation. Key parameters: max_augment_ratio=20 per source demo, max_scene_cycles=3 resampling rounds.

(3B) Cross-degree augmentation (D0→D1). D0 recovery demos are retargeted to D1 error scenes of the same Error Skill by adding a sinusoidal rotation perturbation: magnitude uniformly sampled from $[0.15, 0.5]$ rad along a random axis, temporally smoothed by $\sin(\pi t/T)$. Only the D0→D1 direction is used. Augmented trajectories are validated by execution and success check.

(3C) Cross-subtype augmentation (within-RBG). Source demos from one subtype are retargeted to error scenes of a different subtype within the same RBG. Transfer is capped at 5 augmentations per source demo per target subtype. RBG membership serves as the compatibility criterion: errors within the same RBG share the same canonical subtask ordering, enabling warping without structural mismatch.

Source demo selection. For subtasks referencing a target object, the nearest-neighbor strategy selects from $k=3$ candidates using weighted distance (position weight $\times$ Euclidean + rotation weight $\times$ geodesic). For non-object-referencing subtasks (retract, correct_position), random selection is used.

Table 6: Subtask segmentation thresholds used by the RBG-aware RecoverySegmenter.

Subtask	Segmentation Criterion	Threshold
retract	EEF displacement from start	>0.05 m
navigate_to_object	EEF-to-object distance	<0.06 m
re_orient	Rotation error to target	<0.1 rad
re_grasp	Gripper closure	>0.05
re_lift	End-effector height	>0.84 m
re_transport	Object-to-target distance	<0.05 m
release	Gripper opening	<0.02
correct_position	Position error from intended	<0.03 m

Recovery Behavior Group details. The 12 Error Skills are organized into 5 Recovery Behavior Groups (RBGs) based on the dominant recovery action pattern they require:

- **Re-grasp (A):** Groups E1 (grasp_misalignment) and E11 (grasp_wrong_pose), spanning 4 subtypes. Typically involves retracting, re-orienting, and re-grasping before resuming the task.
- **Retrieve (B):** Groups E2 (drop_in_transit) and E3 (drop_at_wrong_place), spanning 4 subtypes. Centers on navigating back to a dropped or misplaced object.
- **Retract (C):** Groups E4–E7 (collision_holding, collision_empty, collision_eef_object, collision_self), spanning 8 subtypes. Requires backing away from contact and re-approaching safely.
- **Redirect (D):** Contains E8 (wrong_object), spanning 2 subtypes. Requires releasing the wrong object and switching to the correct one.
- **Realign (E):** Groups E9 (trajectory_regression), E10 (stuck_no_progress), and E12 (position_error), spanning 6 subtypes. Focuses on correcting position or task progress before resuming execution.

Redirect is kept as a separate group even though it currently contains only one Error Skill, because recovering from grasping the wrong object requires a qualitatively different recovery pattern from the other groups.

ISO 15066/FMEA mapping. Table 7 maps each Error Skill to the corresponding failure mode categories in ISO/TS 15066:2016 (collaborative robot safety) and standard FMEA (Failure Mode and Effects Analysis) for industrial manipulation systems.

Table 7: Mapping of Error Skills to ISO 15066/10218 hazard categories and FMEA failure modes.

ID	Error Skill	ISO/TS 15066 Category	FMEA Failure Mode	Severity
E1	grasp_misalignment	Incorrect workpiece positioning (§5.4)	Misaligned grasp on workpiece	Moderate
E2	drop_in_transit	Loss of workpiece (§5.3, §5.5)	Object released during transfer	High
E3	drop_at_wrong_place	Loss of workpiece (§5.3, §5.5)	Object deposited at incorrect location	High
E4	collision_holding	Quasi-static/transient contact (§5.5.4)	Collision with environment while holding object	High
E5	collision_empty	Quasi-static/transient contact (§5.5.4)	Collision with environment during approach	Moderate
E6	collision_eef_object	Quasi-static/transient contact (§5.5.4)	End-effector displaces non-target object	Moderate
E7	collision_self	Unexpected robot motion (§5.3.4)	Self-collision or near-singularity contact	High
E8	wrong_object	Incorrect workpiece selection (§5.4)	Gripper acquires non-target object	Moderate
E9	trajectory_regression	Deviation from planned path (§5.3.3)	Task progress regression during execution	Low
E10	stuck_no_progress	Robot standstill / loss of control (§5.3.5)	No measurable progress over time window	Low
E11	grasp_wrong_pose	Incorrect workpiece positioning (§5.4)	Object grasped at wrong orientation	Moderate
E12	position_error	Deviation from planned path (§5.3.3)	End-effector deviates from intended trajectory	Moderate

The mapping follows ISO 10218-1:2011 and ISO/TS 15066:2016 hazard identification frameworks. Collision-related errors (E4–E7) correspond to the “quasi-static and transient contact” hazard class (§5.5.4), which is the primary safety concern addressed by collaborative robot speed and force limiting. Drop errors (E2–E3) correspond to “loss of workpiece” scenarios that industrial FMEA identifies as requiring immediate halt or recovery intervention. Grasp and positioning errors (E1, E8, E11) map to the “incorrect workpiece positioning” category in ISO 10218-1 §5.4, which covers failures in pick accuracy. Trajectory and progress errors (E9, E10, E12) map to “deviation from planned path” (§5.3.3) and “robot standstill” (§5.3.5), representing failures in motion execution rather than physical contact hazards.

Licenses for existing assets. Table 8 lists the external software assets used in this work together with their licenses.

Table 8: Licenses for existing assets used in RECOVERBENCH.

Asset	Version	License
MuJoCo	2.3.7+	Apache 2.0
robosuite	v1.4+	MIT
robomimic	v0.3+	MIT
MimicGen	v1.0	MIT
NumPy	1.24+	BSD-3-Clause
PyTorch	2.0+	BSD-3-Clause
Pi 0.5 base weights (lerobot/pi05_base)	v0.5	Gemma (Google)

All benchmark code, error scene datasets, and evaluation scripts produced as part of this work will be released under the MIT license.

C Per-Task Results

Stack per-subtype breakdown. Table 9 reports per-subtype Recovery Success Rate for Pi 0.5 (Nominal FT) on the stack task. Each subtype is evaluated on 10 validated scenes.

Performance varies substantially across subtypes in this representative case study. `stuck_no_progress_D0`, `trajectory_regression_D0`, and `drop_at_wrong_place_D0` all achieve 100% RSR, suggesting that mild progress interruptions or favorable drop configurations can often be corrected by resuming or re-executing familiar behavior. In contrast, `grasp_wrong_pose_D1` and `position_error_D1` fall to 30%, indicating that severe pose and alignment errors create states much farther from the clean training distribution. The mixed D0/D1 pattern further shows that increasing perturbation magnitude does not always translate monotonically into lower recovery success.

Pick_place per-subtype breakdown. Table 10 reports per-subtype RSR for Pi 0.5 (Recovery) on `pick_place` (best RSR checkpoint: step 19000, overall RSR: 31.67%, CSR: 52%).

Table 9: Per-subtype Recovery Success Rate for Pi 0.5 (Nominal FT) on the stack task (10 trials per subtype).

Subtype	RBG	SR	Subtype	RBG	SR
drop_in_transit_D1	B	60%	collision_eef_object_D0	C	40%
drop_at_wrong_place_D1	B	70%	collision_holding_D0	C	80%
collision_self_D0	C	70%	collision_holding_D1	C	90%
grasp_misalignment_D0	A	70%	collision_eef_object_D1	C	40%
stuck_no_progress_D0	E	100%	grasp_wrong_pose_D1	A	30%
drop_in_transit_D0	B	60%	position_error_D1	E	30%
drop_at_wrong_place_D0	B	100%	wrong_object_D1	D	40%
collision_self_D1	C	70%	trajectory_regression_D0	E	100%
grasp_misalignment_D1	A	50%	collision_empty_D1	C	60%
grasp_wrong_pose_D0	A	70%	position_error_D0	E	60%
collision_empty_D0	C	40%	trajectory_regression_D1	E	70%
stuck_no_progress_D1	E	80%	wrong_object_D0	D	40%

Table 10: Per-subtype Recovery Success Rate for Pi 0.5 (Recovery) on pick_place (10 trials per subtype, step 19000).

Subtype	RBG	SR	Subtype	RBG	SR
grasp_misalignment_D0	A	40%	grasp_misalignment_D1	A	30%
grasp_wrong_pose_D0	A	70%	grasp_wrong_pose_D1	A	20%
drop_in_transit_D0	B	20%	drop_in_transit_D1	B	30%
drop_at_wrong_place_D0	B	50%	drop_at_wrong_place_D1	B	30%
collision_holding_D0	C	20%	collision_holding_D1	C	20%
collision_empty_D0	C	30%	collision_empty_D1	C	80%
collision_eef_object_D0	C	10%	collision_eef_object_D1	C	0%
collision_self_D0	C	10%	collision_self_D1	C	30%
wrong_object_D0	D	30%	wrong_object_D1	D	10%
trajectory_regression_D0	E	0%	trajectory_regression_D1	E	40%
stuck_no_progress_D0	E	30%	stuck_no_progress_D1	E	70%
position_error_D0	E	60%	position_error_D1	E	30%

144 **Coffee per-subtype breakdown.** Table 11 reports per-subtype RSR for Pi 0.5 (Recovery) on coffee
145 (best RSR checkpoint: step 16000, overall RSR: 54.5%, CSR: 84%). Note that coffee exposes 20
146 subtypes (10 Error Skills $\times$ 2 degrees).

Table 11: Per-subtype Recovery Success Rate for Pi 0.5 (Recovery) on coffee (10 trials per subtype, step 16000).

Subtype	RBG	SR	Subtype	RBG	SR
grasp_misalignment_D0	A	70%	grasp_misalignment_D1	A	60%
grasp_wrong_pose_D0	A	60%	grasp_wrong_pose_D1	A	20%
drop_in_transit_D0	B	50%	drop_in_transit_D1	B	40%
collision_holding_D0	C	60%	collision_holding_D1	C	40%
collision_empty_D0	C	70%	collision_empty_D1	C	40%
collision_eef_object_D0	C	50%	collision_eef_object_D1	C	40%
collision_self_D0	C	80%	collision_self_D1	C	20%
stuck_no_progress_D0	E	100%	stuck_no_progress_D1	E	90%
trajectory_regression_D0	E	50%	trajectory_regression_D1	E	50%
position_error_D0	E	70%	position_error_D1	E	30%

147 **Threading per-subtype breakdown.** Table 12 reports per-subtype RSR for Pi 0.5 (Recovery) on
148 threading (best RSR checkpoint: step 19000, overall RSR: 32.0%, CSR: 50%). Threading exposes 20
149 subtypes.

Table 12: Per-subtype Recovery Success Rate for Pi 0.5 (Recovery) on threading (10 trials per subtype, step 19000).

Subtype	RBG	SR	Subtype	RBG	SR
grasp_misalignment_D0	A	60%	grasp_misalignment_D1	A	30%
grasp_wrong_pose_D0	A	20%	grasp_wrong_pose_D1	A	10%
drop_in_transit_D0	B	10%	drop_in_transit_D1	B	10%
collision_holding_D0	C	70%	collision_holding_D1	C	10%
collision_empty_D0	C	40%	collision_empty_D1	C	50%
collision_eef_object_D0	C	0%	collision_eef_object_D1	C	0%
collision_self_D0	C	40%	collision_self_D1	C	0%
stuck_no_progress_D0	E	60%	stuck_no_progress_D1	E	70%
trajectory_regression_D0	E	40%	trajectory_regression_D1	E	20%
position_error_D0	E	70%	position_error_D1	E	30%

150 **Stack_three per-subtype breakdown.** Table 13 reports per-subtype RSR for Pi 0.5 (Recovery) on
151 stack_three (best RSR checkpoint: step 10000, overall RSR: 63.33%, CSR: 52%).

Table 13: Per-subtype Recovery Success Rate for Pi 0.5 (Recovery) on stack_three (10 trials per subtype, step 10000).

Subtype	RBG	SR	Subtype	RBG	SR
grasp_misalignment_D0	A	60%	grasp_misalignment_D1	A	70%
grasp_wrong_pose_D0	A	80%	grasp_wrong_pose_D1	A	30%
drop_in_transit_D0	B	90%	drop_in_transit_D1	B	60%
drop_at_wrong_place_D0	B	70%	drop_at_wrong_place_D1	B	80%
collision_holding_D0	C	90%	collision_holding_D1	C	60%
collision_empty_D0	C	70%	collision_empty_D1	C	50%
collision_eef_object_D0	C	50%	collision_eef_object_D1	C	60%
collision_self_D0	C	50%	collision_self_D1	C	70%
wrong_object_D0	D	70%	wrong_object_D1	D	10%
trajectory_regression_D0	E	40%	trajectory_regression_D1	E	60%
stuck_no_progress_D0	E	70%	stuck_no_progress_D1	E	90%
position_error_D0	E	90%	position_error_D1	E	50%

152 **Three_piece_assembly per-subtype breakdown.** Table 14 reports per-subtype RSR for Pi 0.5
153 (Recovery) on three_piece_assembly (best RSR checkpoint: step 17000, overall RSR: 33.33%, CSR:
154 46%).

Table 14: Per-subtype Recovery Success Rate for Pi 0.5 (Recovery) on three_piece_assembly (10 trials per subtype, step 17000).

Subtype	RBG	SR	Subtype	RBG	SR
grasp_misalignment_D0	A	60%	grasp_misalignment_D1	A	40%
grasp_wrong_pose_D0	A	90%	grasp_wrong_pose_D1	A	30%
drop_in_transit_D0	B	10%	drop_in_transit_D1	B	10%
drop_at_wrong_place_D0	B	10%	drop_at_wrong_place_D1	B	20%
collision_holding_D0	C	50%	collision_holding_D1	C	20%
collision_empty_D0	C	30%	collision_empty_D1	C	0%
collision_eef_object_D0	C	0%	collision_eef_object_D1	C	30%
collision_self_D0	C	50%	collision_self_D1	C	30%
wrong_object_D0	D	10%	wrong_object_D1	D	10%
trajectory_regression_D0	E	50%	trajectory_regression_D1	E	40%
stuck_no_progress_D0	E	50%	stuck_no_progress_D1	E	50%
position_error_D0	E	70%	position_error_D1	E	40%

D Recovery Analysis

Normal SR vs. Recovery SR. The gap between Normal SR and Recovery SR is not simply a byproduct of harder tasks or a different rollout budget. Both metrics evaluate the same policy on the same task under the same success criterion; the only difference is whether execution starts from the nominal initial state or from a validated post-error state. Table 15 reports paired values for Pi 0.5 (Nominal FT).

Table 15: Normal SR vs. Recovery SR for Pi 0.5 (Nominal FT). The performance gap demonstrates that error recovery requires capabilities beyond standard task completion.

Task	Normal SR	Recovery SR
stack	96%	63.3%
pick_place	58%	42.9%
threading	36%	22.5%
three_piece_assembly	28%	17.1%
stack_three	78%	65.0%
coffee	58%	35.5%

D0 vs. D1 severity analysis. We examine whether the D0/D1 severity split behaves monotonically. D1 is intended to represent a larger or more consequential deviation than D0, but lower recovery success should not be assumed a priori. In some cases, a stronger perturbation may partially reset the scene into a familiar approach configuration rather than compounding the difficulty. Table 16 compares matched severity pairs for selected Error Skills on the stack task. Most pairs follow the expected $D0 > D1$ or $D0 = D1$ pattern, and we do not observe a $D1 > D0$ reversal among the selected skills.

Table 16: D0 vs. D1 Recovery Success Rate comparison for Pi 0.5 (Nominal FT) on stack.

Error Skill	D0 SR	D1 SR	Direction
drop_in_transit	60%	60%	$D0 = D1$
drop_at_wrong_place	100%	70%	$D0 > D1$
collision_self	70%	70%	$D0 = D1$
grasp_misalignment	70%	50%	$D0 > D1$
stuck_no_progress	100%	80%	$D0 > D1$

Benchmark comparison. Table 17 positions RECOVERBENCH against representative manipulation benchmarks, data-generation frameworks, and recovery methods.

Table 17: Comparison of RECOVERBENCH with representative manipulation benchmarks, data-generation frameworks, and recovery methods.

Work	Mid-exec. Errors	Rec. Eval.	Shared Tax.	Reprod. Scenes	Primary Focus
<i>Benchmarks / data generation</i>					
LIBERO [?]	No	No	No	No	Lifelong transfer
RLBench [?]	No	No	No	No	Multi-task manipulation
Calvin [?]	No	No	No	No	Long-horizon tasks
THE COLOSSEUM [?]	No	No	No	No	Perturbation robustness
MimicGen [?]	No	No	No	No	Demonstration generation
<i>Recovery methods</i>					
RecoveryChaining [?]	Yes	Partial	No	No	Learned recovery policy
RACER [?]	Yes	Partial	No	No	Recovery-aware IL
FLARE [?]	Yes	Partial	No	No	Recovery skill learning
RESET [?]	Yes	Partial	No	No	Real-world recovery
SC-VLA [?]	Yes	Partial	No	No	Self-correcting VLA
CycleVLA [?]	Yes	Partial	No	No	Backtracking VLA
RECOVERBENCH	Yes	Yes	Yes	Yes	Benchmark